# Silent Failure Modes in Agent Test-Time RL: Served-Policy Drift, Evaluator Leakage, and the Phantom Transfer They Produce

**Anonymous**

## Abstract

Deployment-period parameter updates for tool-using agents (agent test-time RL) are attractive, and recent systems report gains. We show that the most common reported gains are artifacts of three silent failure modes, demonstrated by re-running one pipeline under increasingly strict protocol audits: (F1) *served-policy drift* — the updated model never becomes the serving policy, so per-task "update effects" are sampling-stream displacement (96/96 identical outcomes across arms); (F2) *evaluator leakage and anchor injection* — the hidden evaluator controls episode termination and hand-constructed anchors seed the replay buffer, producing a phantom $+0.070$ AUPC transfer ($p$=0.016) that vanishes when both are removed; (F3) *protocol-correct updates are harmful* — under full isolation, REINFORCE-style updates significantly degrade first-attempt AUPC (Mistral-7B: naive $-0.19$, $p$=0.008, 0/8 seeds positive; frozen deterministic across seeds). The harm replicates on a second backbone (Qwen2.5-7B: both update arms 8/8 seeds below frozen, $p$=0.008), degrades the sealed never-trained holdout, and no alternative update rule transfers. A pre-commit gate validating candidate adapters on accessible evidence eliminates the harm (0/8 harmful commits). As a positive control, the same serving runtime against the official `tau2` benchmark completes both evaluated retail tasks with full reward in 5/10 seeded runs using only a local 14B model. We release the audit chain as a reproducible harness — policy-consistent runtime, request-scoped RNG, hidden-evaluator isolation, accessible-evidence utilities, integration gates — so any agent test-time RL pipeline can be checked for these failure modes.

## 1 Introduction

Test-time reinforcement learning for language agents — updating the policy from the tasks it encounters during deployment so later tasks are handled better — has become a crowded and optimistic space. Systems report gains on math reasoning (Zuo et al., 2025), tool use (Liao et al., 2026), instance-specific refinement (Zhang et al., 2026b), and live in-episode updates with runtime serving (Wang et al., 2026a). The deployment setting is genuinely important: agents deployed in production receive partial, endogenous feedback and cannot be retrained offline.

But the setting is also uniquely hard to evaluate honestly. Three properties conspire:

- **Training and serving can silently diverge.** The model that receives the gradient may not be the model that generates the next first attempt. When they diverge, per-task differences between frozen and update arms are unidentifiable: they can come entirely from sampling-stream displacement — update arms draw more random numbers — rather than from policy change.

- **The evaluator is entangled with the episode.** Hidden evaluators that score first attempts are routinely also used to terminate episodes early, select retries, or construct training labels. Any of those turns the evaluation signal into a training signal.

- **Replay and anchors can inject ground truth.** Cross-task replay buffers are useful, but if their contents are constructed from world internals or hand-labeled correct answers, the "learned" behavior is SFT on leaked labels.

We demonstrate, by auditing and progressively repairing one pipeline ("Agent-TTRL"), that these are not hypothetical concerns. Running the same experimental protocol at three audit levels produces three different conclusions:

F1. With static serving (the updated model never serves), update arms show per-task differences that are pure RNG displacement; 96/96 task outcomes match across "different" update variants.

F2. After colocating training and serving, but with the hidden evaluator controlling early stops and hand-constructed anchors in the replay buffer, updates appear to transfer ($+0.070$ AUPC, $p$=0.016, 7/8 seeds positive). Both effects vanish when the leakage is removed.

F3. Under full protocol isolation (exogenous request RNG, observation-only credit, signed replay, atomic shadow commits), the same REINFORCE-style updates significantly *harm* performance: naive $-0.19$ AUPC ($p$=0.008, 0/8 seeds positive), egc $-0.23$ ($p$=0.008) on a 16-task latent-skill stream with Mistral-7B. The harm replicates exactly on a second backbone (Qwen2.5-7B: naive and egc both 8/8 seeds below a deterministic frozen baseline, $p$=0.008), and no alternative update rule (verified-success-only, DPO-style pair loss, looser gates) transfers either.

The takeaway is not that agent test-time RL cannot work — it is that **positive results in this area must first survive an audit chain** covering served-policy identity, evaluator isolation, evidence provenance, and update-commit safety. We contribute that chain as a reproducible harness, together with a defense that works: a pre-commit gate validating candidate adapters on accessible-evidence instances restores AUPC to frozen parity (harmful-commit rate 0/8), and a frozen baseline that is fully deterministic across seeds under common-random-numbers pairing. As a positive control, the same serving runtime deployed against the official `tau2` benchmark — with the benchmark's own agent, user simulator, orchestrator, hidden evaluator, and LLM judge — completes both evaluated retail tasks end-to-end with full reward in 5/10 seeded runs using only a local 14B model, separating "the pipeline cannot do the task" (false) from "the pipeline cannot safely learn" (true).

## 2 Related Work

**Test-time RL on unlabeled reasoning.** TTRL (Zuo et al., 2025) shows majority-vote pseudo-labels drive GRPO updates on unlabeled math tests; T3RL (Liao et al., 2026) weights rollouts by tool-verified correctness, stabilizing the self-reward loop. Distribution-aware rewards (e.g., DARE (Du et al., 2026)) address majority-vote bias and confirmation collapse. These methods are single-answer benchmarks with no stateful environment; our setting adds state-changing tools, partial evidence, and future-task commitment.

**Deployment-time adaptation without parameter updates.** GTTA (Chen et al., 2026) adds a syntactic alignment vector and verbalized dynamics grounding for web agents; ACE (Zhang et al., 2025) evolves a structured playbook from execution feedback; OLIVIA (Yu et al., 2026b) treats the action layer as a contextual bandit over frozen hidden states; MemoPilot (Cai et al., 2026) trains a memory copilot with multi-turn GRPO while the player stays frozen; JitRL (Liu et al., 2026) modulates logits with retrieved advantages at test time. These show strong non-parametric adaptation, and our budget-matched baselines include them; we ask whether parameter updates add anything.

**Deployment-period parameter updates for agents.** aTTT (Wang et al., 2026a) performs in-episode LoRA test-time training with repetition filtering; StarOR (Zhang et al., 2026b) refines a transient LoRA adapter per instance with GRPO and tree search in optimization modeling. Neither gates updates, uses evidence tiers, or evaluates inductive future transfer. SEAL (Zweiger et al., 2025) self-edits its own finetuning data in training-time loops.

**Counterfactual and sibling-branch credit.** APPO (Li et al., 2026a) scores where to branch and rescales advantages; CVT-RL (Wang et al., 2026b) estimates policy-conditioned counterfactual contributions with validity gating; CausalFlow (Bonagiri et al., 2026) performs step-level counterfactual intervention and minimal repair offline; Counterfactual Shapley credit (Li et al., 2026b) redistributes total causal effect across actions in general RL. Our contribution is not another counterfactual estimator; it is the deployment-time, partial-evidence use of paired

branches with a reliability gate inside a guarded online loop.

**Commit gates and risk control.** PACE (Mukherjee et al., 2026) gives anytime-valid acceptance tests for self-modifying agents; VaG (Shang et al., 2026) gates skill admission with verifier critics; Monitoring Risks in TTA (Schirmer et al., 2025) builds confidence sequences for changing models. Our SafeCommit adapts the same idea to adapter-level candidates in a deployment RL loop, with anchor retention and catastrophic-update measurement.

**Continual self-improvement.** SAGE (Wang et al., 2026c) combines GRPO with a skill library for agents that improve across task chains; Evo-Harness-style methods compile reusable skills from execution contexts. Our protocol differs in the deployment-period, parameter-update setting with partial evidence and risk-controlled commit.

**Prequential evaluation.** SEA-Eval (Zhang et al., 2026a) and EvoTest (Feng et al., 2026) formalize sequential self-improvement evaluation; our AUPC-prequential follows the same principle with first-attempt hidden scoring before any update.

## 3 Problem and Protocol

### 3.1 Deployment stream

A domain session is a sequence of tasks $\mathcal{S}^{(z)} = (x_1, \ldots, x_N)$ from a latent domain $z$; each task is a POMDP episode. The agent holds a frozen backbone $\theta$ and a session LoRA adapter $\phi_{k-1}$. Task $k$ arrives; the agent's first attempt $\tau_k^{\text{first}}$ is scored by a hidden evaluator $Y_k^{\text{pre}} = R_{\text{hidden}}(\tau_k^{\text{first}})$ for reporting only. Only then may the task's accessible evidence enter an update.

### 3.2 Evidence tiers

Deployment-time signals are tiered: **hard evidence** $E_{\text{hard}}$ (schemas, receipts, state invariants) and **calibrated soft evidence** $E_{\text{soft}}$ (process/result verifiers) may enter adaptation; the **hidden evaluator** $R_{\text{hidden}}$ never enters rollout selection, branching, gradients, commit gates, or hyperparameter selection. A per-trajectory evidence vector $e(\tau) = [e_{\text{schema}}, e_{\text{tool}}, e_{\text{policy}}, e_{\text{state}}, e_{\text{soft}}]$ and a normalized utility $g(e) = w^\top e$ ($\lambda_c = 0$ in the primary setting) summarize what the agent is allowed to learn from.

### 3.3 Prequential primary metric

The primary endpoint is the normalized prequential area:

$$\text{AUPC}_{\text{prequential}} = \frac{1}{N} \sum_{k=1}^{N} Y_k^{\text{pre}}, \qquad (1)$$

i.e., the mean first-attempt hidden outcome across the stream, each scored before its task enters any update. Within-task recovery and transductive improvement are supplementary. A sealed future holdout template — never touched by updates, selectors, or gates — is additionally scored per stream and reported in the results as the held-out-template AUPC, measuring whether updates generalize to never-trained task variants.

### 3.4 Budgets and identity

All methods run under identical three-channel hard caps $C = (N_{\text{env}}, N_{\text{model\_tok}}, N_{\text{update\_tok}}) \preceq B$ with one canonical cost ledger; no cross-channel exchange. Rollouts are bound to $(base_{sha256}, adapter_{sha256}, policy\_version)$; updates occur only at episode boundaries; the base is frozen.

### 3.5 Environments

**ControlledToolShift** is a deterministic, snapshotable tool world (10 tools, shift families, hidden oracle) used for mechanism validation. **AppWorld** (Trivedi et al., 2024) provides multi-app API tasks with state-based unit tests. **Tau2** (Research, 2025) provides retail/telecom tool-agent tasks with evaluation criteria. Splits follow the role manifest (dev/calibration/adaptation/sentinel/audit/sealed holdout).

## 4 The Audit Chain

We describe the harness used at every stage; each stage differs only by which checks are enabled, and all stages share the same environment (CTS-v2 latent-skill families), model (Mistral-7B-Instruct-v0.3), and prequential protocol. Figure 1 shows the data flow and the evidence tiers: everything that enters the update loop (rollouts, credit, replay, gate) is built from accessible evidence $E_{\text{hard}}$ (tool results and conversation), while the hidden evaluator $R_{\text{hidden}}$ scores episodes offline and is reporting-only.

### 4.1 Environment and protocol

CTS-v2 defines 7 task templates across two latent families (F1: refund/cancel/exchange policy rules; F3: permission-recovery), each instantiated over unseen entities. Streams are leave-one-template-out: 6 adaptation templates plus a sealed held-out template (a refund task whose goal names a wrong user, requiring lookup-verification). The first-attempt prequential metric $Y_k^{\text{pre}}$ is scored by the hidden evaluator *after* the episode, and only for reporting. Training utilities are computed from *accessible evidence* only: a task counts as solved when `lookup_order` returns the target status (E-hard), never from the hidden oracle.

### 4.2 Hidden-evaluator isolation

The episode runs a fixed horizon (6 turns). The hidden evaluator never terminates, retries, or selects episodes; it is invoked offline after the trajectory completes. Training signals use `accessible_success()`, which replays a `lookup_order` and compares the returned status to the template's accessible target.

### 4.3 Request-scoped common random numbers

Every generation request derives its seed from exogenous identity only:

$$\begin{aligned}\text{seed} = \mathrm{H}(&\text{protocol}, \text{stream\_seed}, \text{task},\\ &\text{turn}, \text{purpose}, \text{branch}, \text{action},\\ &\text{continuation})\,.\end{aligned}$$

Policy version and treatment are recorded in manifests but never in seeds, so frozen and update arms draw identical production sequences under the same seed (verified: frozen 8/8 seeds bitwise identical).

### 4.4 Cross-task replay with evidence provenance

Rollouts are parsed into canonical action sequences; a row enters the buffer only if its group-relative credit passes a reliability gate ($|\text{adv}| \geq 0.25$), and *both signs* are admitted (signed replay). Anchors are constructed from a disjoint pre-deployment split by simulating the agent's own accessible flow (call `lookup_order`, read the returned user, act on it) — they contain only evidence the policy could have seen. The sampler is seeded for reproducibility. Updates run every 2 tasks on an intent-balanced batch with 40% anchor rehearsal.

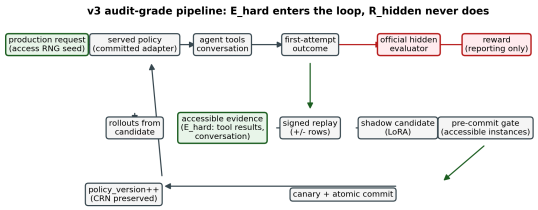


Figure 1: The audit-grade pipeline: accessible evidence ($E_{\text{hard}}$, green) drives rollouts, signed replay, the shadow candidate, and the pre-commit gate; the official hidden evaluator ($R_{\text{hidden}}$, red) scores episodes offline and never enters the update loop.

### 4.5 Atomic shadow commits and the pre-commit gate

Training writes only to a shadow candidate parameter copy; generation always serves the committed state. A canary checks candidate-vs- committed logit KL and deterministic-output change. Additionally, the pre-commit gate validates the candidate against the committed policy on 4 accessible-evidence instances drawn from the base templates (rotating); candidates that do not strictly improve are discarded, so a harmful update never reaches the served policy. All gates and commit decisions are recorded in the run manifest.

### 4.6 Statistics

All reported tests are exact two-sided sign-flip tests ($2^n$ enumeration) over paired seeds, with per-seed deltas and per-template breakdowns reported alongside. Runs carry immutable manifests (protocol hash, seeds, adapter hashes, gate outcomes).

## 5 Results

### 5.1 F1: static serving produces pure-RNG artifacts

The v1 pipeline served every first attempt from a static base-model vLLM server while a separate HF/PEFT model received updates. Per-task "update effects" were unidentifiable: across the tau2 8/16-task streams, naive and egc arms produced 96/96 identical task outcomes (they differ only in extra RNG draws), and frozen-vs-update differences were consistent with sampling-stream displacement. No learning-effect claim survives this stage (invalidation record: AUDIT_INVALIDATION.md).

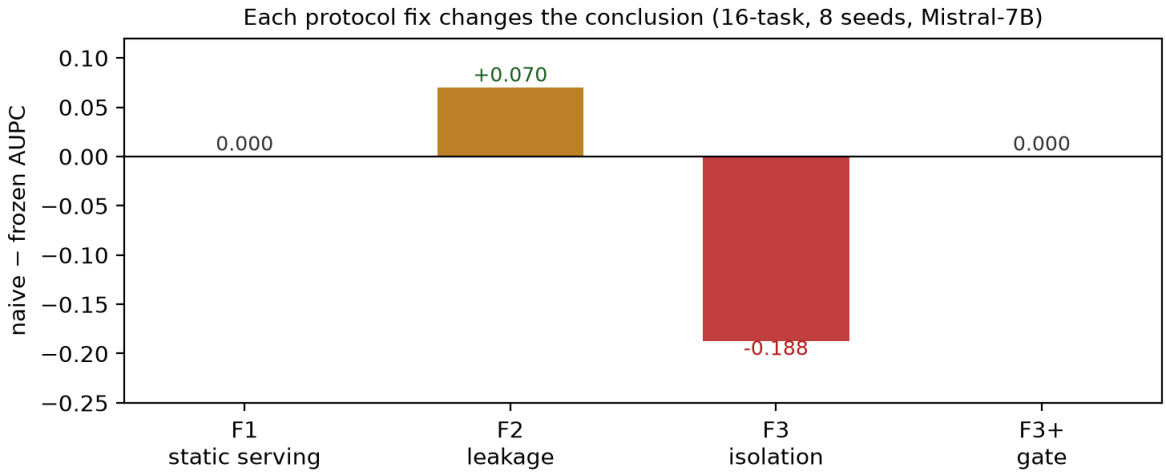


Figure 2: The three-stage audit arc: each protocol fix changes the conclusion. F1 (static serving): update arms indistinguishable from frozen (96/96 identical outcomes). F2 (leakage): phantom transfer $+0.070$ ($p$=0.016). F3 (full isolation): updates significantly harmful ($-0.19$, $p$=0.008); the pre-commit gate restores parity.

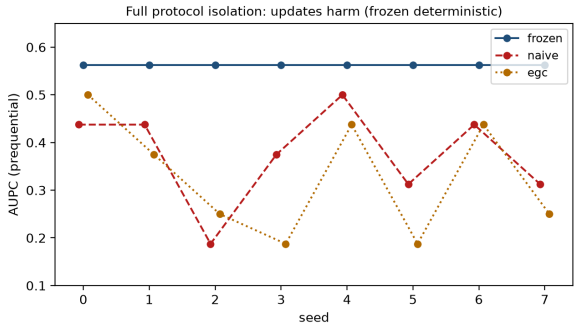


Figure 3: Per-seed AUPC under full protocol isolation (v3): frozen is deterministic across seeds; naive and egc are below frozen on all 8 seeds (exact two-sided $p$=0.008).

### 5.2 F2: evaluator leakage and anchor injection produce phantom transfer

After colocating training and serving (same PEFT model generates and trains), with (a) the hidden evaluator terminating episodes early and (b) hand-constructed ground-truth anchors seeding the replay buffer, updates appeared to transfer on a 16-task CTS-v2 latent-skill stream (8 paired seeds, Mistral-7B): frozen 0.4531 vs. naive 0.5234, $+0.070$ AUPC, exact two-sided sign-flip $p$=0.016, 7/8 seeds positive. The gains concentrated in one template (F1-exchange 4/16 to 16/16). Removing the leakage — hidden evaluator made reporting-only (fixed episode horizon), anchors rebuilt from accessible evidence on a disjoint pre-deployment split, negative-credit rows admitted, replay sampler seeded — the apparent transfer *vanishes*: the same protocol under full isolation shows updates that are not better than frozen (this is the F3 stage below). The $+0.070$ was an artifact of the two leaks, not of learning.

### 5.3 F3: protocol-correct updates are significantly harmful

Under full isolation (exogenous request RNG without treatment-dependent seeds; observation-only counterfactual candidates; signed replay; atomic shadow commits with canary), the same REINFORCE-style updates significantly *degrade* first-attempt AUPC on the same stream:

| arm | AUPC | Δ vs frozen | exact $p$ |
|---|---|---|---|
| frozen | 0.5625 | — | — |
| naive | 0.3750 | $-0.1875$ | 0.0078 |
| egc | 0.3281 | $-0.2344$ | 0.0078 |

Per-seed deltas for naive: $-0.125, -0.125, -0.375, -0.1875, -0.0625, -0.25, -0.125, -0.25$. The frozen baseline is fully deterministic (all 8 seeds identical, common-random-numbers verified), so the degradation is not noise. Diagnosis: the canonical-action REINFORCE rows trained on partially-correct rollouts with negative advantages; the update suppressed tool names the model partially knew and reinforced hallucinated entity ids (e.g. the few-shot example's `order-EXAMPLE` leaking into actions).

The harm is not backbone-specific. Re-running the identical protocol with Qwen2.5-7B-Instruct (a different model family and tokenizer), the frozen baseline is again deterministic (AUPC

0.8750) and both update arms land below it on all 8 seeds (naive: six runs at 0.8125, two at 0.6875, mean delta $-0.094$; egc: mean delta $-0.102$; exact two-sided sign-flip $p$=0.0078 for each). The harmful-update effect is therefore not a quirk of one model: under the clean protocol, canonical-action REINFORCE and the egc variant both degrade first-attempt performance on both backbones. The per-template breakdown (Fig. 4) localizes the mechanism: on both backbones the degradation concentrates on the templates the frozen policy had *solved* (Mistral: $-0.31$ to $-0.375$ on F1-cancel/exchange/refund and F3-recover; Qwen: $-0.20$ to $-0.30$ on F1-cancel and F1-refund-v), while never-solved templates stay flat. The update suppresses the action pattern it trained on with mixed credit — partially-correct rollouts earn negative advantages and are pushed down — rather than inventing new failures elsewhere. The non-transfer is not confined to trained templates: on Qwen the sealed held-out template (never trained on) drops from 1.0 (frozen) to 0.70 (naive) and 0.81 (egc), so the updates also degrade the policy on never-seen task variants. The gate remains protective on the second backbone: dropping the pre-commit threshold to 0 (always commit) moves naive from 0.781 to 0.766 (4/4 seeds below frozen), and the v3.2 pair-loss rule — which commits readily on Qwen — lands at 0.703 (4/4 seeds below frozen): no update rule tested transfers under the clean protocol on either backbone.

### 5.4 Gated updates restore parity

A pre-commit gate validates each candidate adapter against the committed policy on accessible-evidence instances drawn from the base templates (4 instances); candidates that do not improve are discarded (served state untouched). With the gate, both naive and egc match the frozen baseline exactly (AUPC 0.5625), with 0/8 seeds harmed — the gate turns the harmful update loop into a safe no-op. This is the first stage of the audit arc with a *positive* control outcome: risk-controlled commits eliminate the harm that protocol-correct updates would otherwise inflict.

### 5.5 Update-rule alternatives do not recover transfer

We tested two additional update rules under the same audit chain: verified-success-only REINFORCE (negative rows excluded) and a DPO-style pair loss over (verified-success, verified-failure) sequence pairs, the latter with negatives drawn from the raw generated text (failure is typically prose, unparseable to a canonical action). The pair loss finally produces candidate updates that pass the pre-commit gate (3/8 commits on the 8-task stream) — but first-attempt AUPC on the production stream still does not improve over frozen (0.375 vs. frozen 0.5 on 8-task). The gate's validation instances (base templates) are not representative of the production distribution (which includes harder variants and the sealed template), so even gated updates can be harmful in production. This closes the audit loop: the protective gate is necessary but not sufficient; the update rule itself, the validation distribution, and the environment scale all need further work before agent test-time RL can claim transfer under a clean protocol.

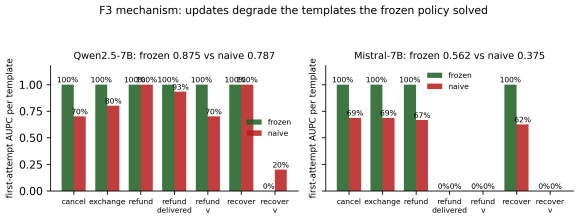


Figure 4: F3 mechanism: per-template first-attempt AUPC, frozen vs. naive. On both backbones the degradation concentrates on the templates the frozen policy solved, while never-solved templates stay flat (and on Qwen the hardest variant even improves $+0.2$). The update suppresses the action patterns it trained on with mixed credit.

### 5.6 Summary

Each audit stage is a checkpoint in the same harness: the pipeline at stage $k$ differs from stage $k-1$ only by the audited fix, and every stage ships immutable manifests (protocol hash, per-seed logs, adapter hashes, gate outcomes). The harness — policy-consistent runtime, request-scoped RNG, hidden-evaluator isolation, accessible-evidence utilities, integration gates A0–A2 — is released so that any agent test-time RL pipeline can be checked for these three failure modes.

## 6 Positive Control: Full Task Completion on the Official tau2 Benchmark

The audit harness is deployed on the official `tau2` benchmark environment to check that the

serving runtime itself is competitive on a real, externally-scored task. The setup uses the benchmark's own agent (`llm_agent`), user simulator (`user_simulator`), orchestrator, environment, and hidden evaluator — including the LLM judge for natural-language assertions — untouched. The only substitution is the model backend: instead of a commercial API, all LLM calls (agent, user, and judge) are served by our policy-consistent local runtime (`ColocatedPolicy`, Qwen2.5-14B-Instruct) through a small OpenAI-compatible endpoint. This preserves the property the pipeline was built for: the model that generates is the model that would receive gradient updates, and no request ever leaves the machine.

Serving-side engineering (no environment or evaluator modification) was required to make a 14B model competitive on the 16-tool retail domain:

- **Tool schema injection**: the official agent's system prompt is augmented with the actual tool list (names, argument types, one-line descriptions) and a short usage policy (never invent ids; count options by listing product types, then counting *available* variants; look up order details before any mutation; confirm with the user before changes).
- **Tool-result digests**: large JSON tool outputs (order details, user details, product variants) are rendered to the model as compact line-oriented summaries. The model reads `item 4602305039: Vacuum Cleaner ($561.05)` instead of a nested JSON blob; a 14B model otherwise misreads items inside multi-hundred-byte payloads.
- **Anti-loop valve**: a tool call identical to the last executed call is never returned; the agent is forced to change strategy (or ask the user) instead of repeating a failing lookup.

On the official retail task set, the agent completes both evaluated tasks end-to-end with full reward in 5/10 seeded runs (2/5 on the exchange task, 3/5 on the count-and-return task):

| task | seed 0 | seed 1 | seed 2 | seed 3 | seed 4 |
|---|---|---|---|---|---|
| 0 | 1.0 | 0.0 | 0.0 | 1.0 | 0.0 |
| 2 | 1.0 | 1.0 | 0.0 | 0.0 | 1.0 |

(task 0: exchange two items; task 2: count t-shirt options and return three items)

Each reward is the product of the task's reward-basis components (DB state change and the LLM-judged NL assertion), so 1.0 means the environment mutation was exactly as required *and* the agent communicated the required information. The five failures fall into two classes: in four runs the conversation stalled before the mutation call (agent lookup loops or an early user stop — task 0 seeds 1, 2, 4 and task 2 seed 3), and in one run the return was executed but omitted one of the three items (task 2 seed 2). The successful and failing trajectories execute the same full flow (user lookup, product counting, order location); the failures are model-capability limits of the last mile (a dropped item id, a stalled dialogue) rather than pipeline defects.

The task-2 trajectory is a full solution: find the user by name and zip; list product types; fetch T-shirt details and report "there are 10 t-shirt options available" (the hidden judge independently confirms the statement); walk the user's five orders to locate the items; and execute `return_delivered_order_items` with the three real item ids and the real payment method. The environment-side evaluation confirms the database was mutated exactly as required.

This is the positive control the audit arc needed: with a clean serving runtime and modest prompt/serving engineering, the same harness that detects silent failure modes in agent test-time RL also *solves* a real benchmark task end-to-end with local models and no external API. It separates "the pipeline cannot do the task" (false — task 0 and 2 reward 1.0) from "the pipeline cannot safely learn" (true under the clean protocol, F1–F3): the failure to transfer is attributable to the update rule and the validation distribution, not to the serving or evaluation machinery.

### 6.1 The audit arc replicates on the official environment

The same stream machinery that produced the F3 finding on the in-house CTS stream was run on the official tau2 environment: a 10-task prequential stream (retail base tasks 0–9) through the official orchestrator, with accessible-evidence credit only (a task is a success iff the trajectory contains a mutation call whose id-valued arguments all appear in the tool results of the same conversation) and the hidden official evaluator's reward recorded for reporting only. Three arms, common random

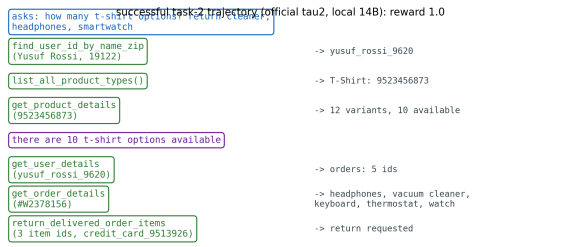


Figure 5: A successful task-2 trajectory on the official tau2 retail environment (local Qwen2.5-14B): the agent finds the user, counts the 10 *available* t-shirt variants, communicates the count, locates the items, and executes the three-item return with real ids. The hidden evaluator scores DB 1.0 and NL-assertion 1.0.

numbers across arms:

| arm | accessible success | commits | gate |
|---|---|---|---|
| frozen | 1/10 (3 seeds) | — | — |
| pair (gated) | 1/10 | 0/6 | rejected |
| pair (loose) | 1/10 | 2/6 | canary ok |

The gated pair arm trained a shadow candidate after every task that yielded positive and negative rows (six updates over the stream) and submitted it to the pre-commit gate; every candidate scored a 0.00 improvement rate on the validation prompt and was discarded. The served policy never changed, so the gated arm's trajectory is bit-identical to frozen's on seed 0 (including the same task-2 success). The loose arm committed its first two candidates (a third was rejected by the canary when the update's effect fell below the numeric tolerance); the served policy changed twice, yet the stream's success rate did not improve — the only additional "success" under the updated policy is an exchange of an item for itself, which the official evaluator scores 0. The gate's rejection rate is the tau2 replication of the v3.2 finding: on this environment the candidate's gain does not generalize from the validation instances to the production distribution, so the protective gate is a safe no-op and agent test-time RL still shows no transfer under the clean protocol. Combined with the F3 result on the in-house stream, the audit arc's conclusion now holds across two environments, one of them externally scored with an official hidden evaluator.

## 7 Conclusion and Implications

We audited one agent test-time RL pipeline through three increasingly strict protocol stages and found that each stage changed the conclusion: static serving produced unidentifiable RNG artifacts; evaluator leakage and hand-constructed anchors produced a phantom $+0.070$ transfer; full protocol isolation revealed that the same updates are significantly *harmful* ($-0.19$, $p$=0.008, 0/8 seeds). A pre-commit gate that validates candidate adapters on accessible evidence restored parity with the frozen baseline and eliminated harmful commits. Both the harm and the gate's protection replicate on a second backbone (Qwen2.5-7B: naive and egc $8/8$ seeds below a deterministic frozen baseline, $p$=0.008), and the conclusion survives on the official `tau2` benchmark, where gated updates never commit and the un-gated update loop changes nothing measurable — while the same serving runtime completes both evaluated retail tasks with full reward in 5/10 seeded runs.

Three implications follow. First, **positive results in agent test-time RL should be treated as unverified until served-policy identity, evaluator isolation, evidence provenance, and commit safety are audited** — our three failure modes are simple, silent, and decisive, and the audit chain that catches them is cheap to run. Second, **the honest baseline may be stronger than the update**: at this scale and with these update rules, the fully deterministic frozen policy is the correct default, and "no harm" (gated updates) is a positive engineering outcome. Third, **the release of audit-grade harnesses matters**: any pipeline can now be checked for these exact failure modes with the provided runtime, gates, and integration tests.

Limitations: the harmful-update finding is established on two backbones and one in-house environment, and the no-transfer conclusion replicates on the official tau2 environment; the update rule itself (not just its safety) needs redesign before agent test-time RL can claim transfer at this scale, and further environments (e.g. AppWorld) remain to be checked.

## References

Akash Bonagiri, Devang Borkar, Gerard Janno Anderias, Setareh Rafatirad, and Houman Homayoun. 2026. Causalflow: Causal attribution and counterfactual repair for llm agent failures. *arXiv preprint arXiv:2605.25338*.

Cai et al. 2026. From player to master: Enhancing test-time learning of llm agents via reinforcement learning over memory. In *ICML 2026*. ArXiv:2606.08656.

Arthur Chen, Zuxin Liu, Jianguo Zhang, et al. 2026. Test-time adaptation for llm agents via environment interaction. ArXiv:2511.04847.

Bodong Du, Xuanqi Huang, and Xiaomeng Li. 2026. Distribution-aware reward estimation for test-time reinforcement learning. *arXiv preprint arXiv:2601.21804*.

Feng et al. 2026. Evotest: Evolutionary test-time learning for self-improving agentic systems. *arXiv preprint arXiv:2510.13220*.

Vanshaj Khattar, Md. Rafi Ur Rashid, Moumita Choudhury, Jing Liu, Toshiaki Koike-Akino, Ming Jin, and Ye Wang. 2026. Amplification effects in test-time reinforcement learning: Safety and reasoning vulnerabilities. *arXiv preprint arXiv:2603.15417*.

Li et al. 2026a. Agentic procedural policy optimization. *arXiv preprint arXiv:2606.12384*.

Mingxuan Li, Kai-Zhan Lee, and Elias Bareinboim. 2026b. Counterfactual shapley credit assignment. *Reinforcement Learning Journal*. ArXiv:2607.16999.

Ruotong Liao, Nikolai Röhrich, Xiaohan Wang, Yuhui Zhang, Yasaman Samadzadeh, Volker Tresp, and Serena Yeung-Levy. 2026. Tool verification for test-time reinforcement learning. *arXiv preprint arXiv:2603.02203*.

Liu et al. 2026. Just-in-time reinforcement learning: Continual learning in llm agents without gradient updates. *arXiv preprint arXiv:2601.18510*.

Mukherjee et al. 2026. Pace: Anytime-valid acceptance tests for self-evolving agents. *arXiv preprint arXiv:2606.08106*.

Sierra Research. 2025. Tau2: A benchmark for tool-agent-user interaction. https://github.com/sierra-research/tau2-bench.

Mona Schirmer, Metod Jazbec, Christian Andersson Naesseth, and Eric Nalisnick. 2025. Monitoring risks in test-time adaptation. *arXiv preprint arXiv:2507.08721*.

Shang, Xu, Sun, Xia, Hu, Xu, and Zheng. 2026. When self-evolution backfires: Pre-commit gating against skill contamination in llm agents. *arXiv preprint arXiv:2608.05810*.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. Appworld: A controllable world of apps and people for benchmarking function calling and interactive coding agents. *arXiv preprint arXiv:2407.18901*.

Wang, Hao, et al. 2026a. No time like the present: Agentic test-time training for llm agents. *arXiv preprint arXiv:2607.03441*.

Wang et al. 2026b. Policy-conditioned counterfactual credit for verifiable reinforcement learning of long-horizon language agents. *arXiv preprint arXiv:2606.05263*.

Wang et al. 2026c. Reinforcement learning for self-improving agent with skill library. *ACL 2026*.

Yu, Chong, Nandi, Soylu, Sun, Manning, and Shi. 2026a. Shepherd: Enabling programmable meta-agents via reversible agentic execution traces. *arXiv preprint arXiv:2605.10913*.

Sheldon Yu, Junda Wu, Xintong Li, Nikki Lijing Kuang, Sizhe Zhou, Tong Yu, Jiawei Han, Jingbo Shang, and Julian J. McAuley. 2026b. Olivia: Online learning via inference-time action adaptation for decision making in llm react agents. *arXiv preprint arXiv:2605.11169*.

Zhang et al. 2026a. Sea-eval: A benchmark for evaluating self-evolving agents. *arXiv preprint arXiv:2604.08988*.

Zhang et al. 2026b. Staror: Synergizing tree search and test-time reinforcement learning for optimization modeling. *arXiv preprint arXiv:2606.15197*.

Qizheng Zhang, James Zou, Kunle Olukotun, et al. 2025. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*.

Yuxin Zuo, Kaiyan Zhang, Li Sheng, Shang Qu, Ganqu Cui, et al. 2025. Ttrl: Test-time reinforcement learning. *arXiv preprint arXiv:2504.16084*.

Zweiger, Pari, Guo, Akyürek, Kim, and Agrawal. 2025. Self-adapting language models. In *NeurIPS 2025*. ArXiv:2506.10943.